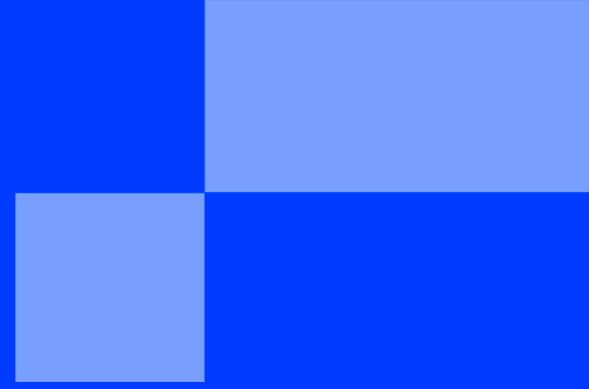




ALICE'S ADVENTURES IN WONDERLAND

< NESTED LITERARY PUZZLES />



ALICE'S ADVENTURES IN WONDERLAND

Like the *White Rabbit*, most people are always running out of time.

You've joined *Through the Looking-Glass* — a startup building a **lightweight** tool that creates "book cards" to help publishers and editors quickly make sense of *Wonderland's* endless library, without having to read the books entirely.



They challenge you to develop a **prototype NLP engine** performing operations on books from [Project Gutenberg](#) through dedicated **CLI** commands.

Just as the *Caterpillar's* mushrooms transformed Alice's perception of the world, your tool will transform raw text into structured understanding — turning long books into small cards.



Some operations may be computationally expensive.
You should save their results to reuse them later.

NLP tasks



The bootstrap should help you tremendously to implement some atomic features that you'll be able to reuse across your project pipelines.

In order to craft a book card, you'll need to implement the common NLP tasks below.



For each task, you **MUST** illustrate its pipeline main steps on a separate diagram.

Lexical diversity measurements

Your program evaluates how rich and varied the vocabulary of a book is. This helps to reveal an author's linguistic style, which is useful for targeting specific audiences or evaluating writing quality.

The `--lexdiv <ID>` option returns various book metrics into a dictionary:

```
{
  "tok":int,      # total number of word tokens
  "typ":int,      # number of unique word tokens
  "hap":int,      # number of word tokens occurring only once
  "ttr":float,    # number of unique words tokens divided by number of word tokens
  "mwl":float,    # mean number of characters per word token
  "mwf":float     # number of word token divided by number of unique word tokens
}
```



We're all mad here. I'm mad. You're mad.

— The Cheshire Cat —

Topic modeling

Your program uncovers the main themes that emerge across a book. This helps editors quickly understand what a book is about, and assists publishers in organizing large catalogs by thematic categories.

The `--topics <ID>` option extracts the main topic top 10 words **from each section** of a book into a dictionary, which respects the following pattern `{1: list[str], 2: list[str], ...}`

Named Entities Recognition

Your program identifies characters and locations mentioned in the book, in order to ease cross-referencing between books, and the creation of character maps or timelines.

The `--entities <ID>` option extracts the locations and characters of a book in a dictionary, which respects the following pattern `{"characters": list[str], "locations": list[str]}`.



Why is a raven like a writing-desk?

— The Hatter —

Book summarization

Your program condenses the book into short summaries, in order to present concise overviews on catalogs, and provide a quick preview of the books to the readers.

The `--summarize <ID>` option returns a string summarizing a book in a string of few sentences.



Various methods exist to summarize a text. Choose the most suited one.

Books similarity

Your program compares books to identify stylistic or thematic resemblances, in order to build a recommendation system for the readers.

The `--similar <ID>` option returns a list of 5 similar books, such as `["title1", ..., "title5"]`. This list should be sorted by decreasing similarity.

A way of doing this is to vectorize multiple books and to compute cosine similarity. You should work on the following book collection:

ID	Title	Category
11	Alice's Adventures in Wonderland	Children / Young Adult
12	Through the Looking-Glass	Children / Young Adult
16	Peter Pan	Children / Young Adult
55	The Wonderful Wizard of Oz	Children / Young Adult
113	The Secret Garden	Children / Young Adult
120	Treasure Island	Children / Young Adult
236	The Jungle Book	Children / Young Adult
108	The Return of Sherlock Holmes	Crime, Mystery & Thriller
834	The Memoirs of Sherlock Holmes	Crime, Mystery & Thriller
863	The Mysterious Affair at Styles	Crime, Mystery & Thriller
1661	The Adventures of Sherlock Holmes	Crime, Mystery & Thriller
61262	Poirot Investigates	Crime, Mystery & Thriller
69087	The murder of Roger Ackroyd	Crime, Mystery & Thriller
70114	The Big Four	Crime, Mystery & Thriller
35	The Time Machine	Science-Fiction & Fantasy
36	The War of the Worlds	Science-Fiction & Fantasy
84	Frankenstein; Or, The Modern Prometheus	Science-Fiction & Fantasy
159	The island of Doctor Moreau	Science-Fiction & Fantasy
164	Twenty Thousand Leagues under the Sea	Science-Fiction & Fantasy
345	Dracula	Science-Fiction & Fantasy

Book card

Finally, your program compiles all the previously extracted information into a compact, structured format, to facilitates the creation of "metadata" sheets providing anat-a-glance guide to understand the essence of a book before diving in.

The `--card <ID>` option gathers information about a book and returns a dictionary that should look like this:

```
{
  "info": {"id": str, "authors": str, "bookshelves": str},
  "lexdiv": {"tok": int, "typ": int, "hap": int,
            "ttr": float, "mwl": float, "mwf": float},
  "topics": {1: list[str], ..., 4: list[str]},
  "entities": {"characters": list[str], "locations": list[str]},
  "summary": str,
  "similar": ["title1", ..., "title5"],
}
```



You're nothing but a pack of cards!

— Alice —

Expectations

You must deliver, present and demonstrate a script named `bookworm.py`.

You should also respect these requirements:

- ✓ Your code must:
 - follow clean coding standards;
 - comply with the specified CLI nomenclature.
- ✓ Your program must:
 - handle errors gracefully and provide clear messages;
 - run seamlessly on any machine (not only yours);
 - not use any heavy models in order to run fast without consuming too many resources.
- ✓ Your repository must:
 - include frequent commits with clear and descriptive messages;
 - be well-organized, with a clean folder structure and no redundant files;
 - exclude unnecessary or large files and folders;
 - contain documentation describing the project, setup instructions, and usage examples.

Bonuses

Below is a non-exhaustive list of ideas to help you take this project further:

- ✓ retrieve information and download all books [co-]written by a given author;
- ✓ retrieve information and download all books from a specific category;
- ✓ fetch book covers to illustrate your book cards;
- ✓ generate a word cloud of the most frequent words to illustrate your book cards;
- ✓ compute additional lexical diversity measurements such as: MSTTR, MATTR, MTLD, Evenness, Rarity, Dispersion, Disparity, Guiraud's Index, Herdan's C, Yule's I, Yule's K, Maas's a^2 , Dugast's U^2 ;
- ✓ compare topic modeling results using LSA and LDA, and choose the best approach;
- ✓ export book cards in a cross-platform and user-friendly format;

- ✓ cluster books using K-Means or hierarchical clustering to group similar texts;
- ✓ build a simple frontend to make your NLP engine interactive.

v 1.5

{EPITECH}